Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

CS7IS2 - Artificial Intelligence

Assignment 1: Search and Markov Decision Processes

Comparative Analysis of Search Algorithms and Markov Decision Processes

on Randomly Generated Mazes

Author	Naman Kumar
Student ID	25340053
Module	CS7IS2 — Artificial Intelligence
Institution	Trinity College Dublin
Date	1st March 2026

Contents

1 Introduction **2**

2 Implementation **2**

2.1 Maze Generation 2

2.1.1 Perfect vs Imperfect Mazes 2

2.2 Search Algorithms 3

2.2.1 Breadth-First Search (BFS) 3

2.2.2 Depth-First Search (DFS) 4

2.2.3 A^* Search 5

2.3 Markov Decision Processes 7

2.3.1 Value Iteration (MDP-VI) 7

2.3.2 Policy Iteration (MDP-PI) 7

3 Experimental Setup **9**

4 Results: Search Algorithm Comparison **10**

4.1 Unweighted Maze, Raw Data 10

4.2 Path Quality 10

4.3 Exploration Efficiency 11

4.4 Wall-Clock Time and Per-Node Overhead 12

4.5 Weighted Maze, Search Algorithm Comparison 12

5 Results: MDP vs Search Comparison (Unweighted) **13**

5.1 Raw Benchmark Data 14

5.2 Path Quality: MDP Matches Search 14

5.3 The Speed Gap 15

5.4 MDP-VI vs MDP-PI 15

5.5 Time Complexity Analysis 16

5.6 When Does MDP Make Sense? 16

6 Conclusions **17**

A Appendix **20**

A.1 Maze Generation and Neighbour Lookup 20

A.2 Breadth-First Search (BFS) 21

A.3 Depth-First Search (DFS) 21

A.4 A* Search 22

A.5 MDP, Stochastic Transition Model 23

A.6 MDP, Value Iteration 23

A.7 MDP, Policy Iteration 24

1 Introduction

Pathfinding in discrete, grid-based environments is a classical problem in artificial intelligence. Search algorithms and Markov Decision Process (MDP) solvers represent two fundamentally different paradigms: search explores a graph reactively from a known start state, while an MDP computes a complete policy over the entire state space before any action is taken.

This report implements, benchmarks, and compares six algorithms on randomly generated maze instances of varying sizes (5×5 to 100×100). The maze generator is `pyamaze`[1], configured with `loopPercent = 50` to produce multiply-connected environments that stress-test both heuristic guidance and policy convergence.

The algorithms are:

- **BFS**, Breadth-First Search (uninformed, step-optimal)
- **DFS**, Depth-First Search (uninformed, non-optimal)
- **A* (Manhattan)**, informed search with Manhattan heuristic
- **A* (Euclidean)**, informed search with Euclidean heuristic
- **MDP-VI**, Value Iteration ($\gamma = 0.9$, noise $= 0.2$)
- **MDP-PI**, Policy Iteration ($\gamma = 0.9$, noise $= 0.2$)

Additionally, **Weighted A*** variants (A*W) are evaluated in the weighted maze scenario, where cell traversal costs vary from 1 to 9.

All algorithms navigate from the bottom-right corner (N, N) to the top-left corner $(1, 1)$. The study is organised into three comparison axes: (i) search algorithm comparison on unweighted and weighted mazes, (ii) MDP vs search comparison on unweighted mazes, and (iii) stochastic robustness analysis.

2 Implementation

2.1 Maze Generation

Mazes are generated using `pyamaze`'s `CreateMaze(loopPercent=50)` call. The maze is represented as a dictionary `maze_map[{(row, col)}][d]` mapping each cell and direction to a binary wall indicator (1 = open, 0 = wall). For each size, **one maze is generated** and *all algorithms* are executed on that same maze, ensuring fair head-to-head comparison.

2.1.1 Perfect vs Imperfect Mazes

A **perfect** maze is a spanning tree of the grid: exactly one path between any pair of cells, no cycles[10]. Setting `loopPercent=50` removes roughly half the remaining internal walls, producing an **imperfect** (multiply-connected) maze with cycles and multiple alternative routes. This has two important consequences: (1) different algorithms can find genuinely different routes, making comparisons meaningful; (2) MDP policies can exploit alternative paths for recovery under stochastic transitions — a mechanism

unavailable in perfect mazes. However, cycles also increase the iterations required for Bellman convergence.

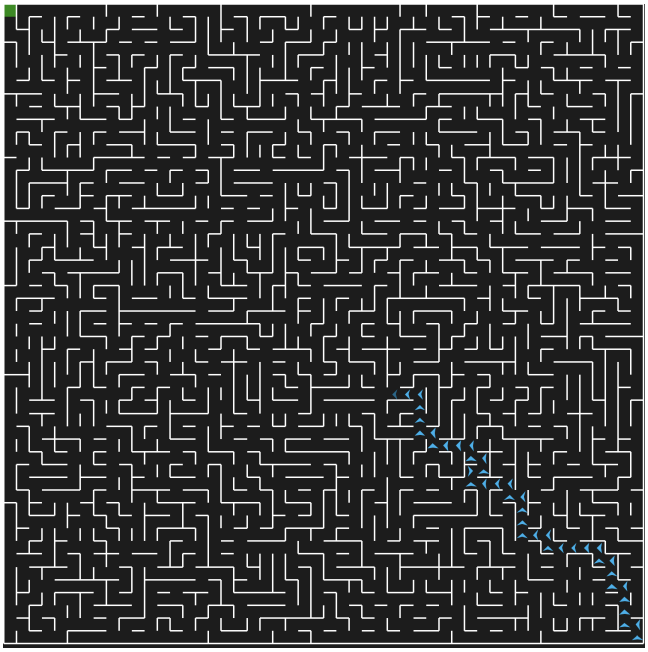


Figure 1: Sample maze with `loopPercent=50`. Start: (N, N) bottom-right; Goal: $(1, 1)$ top-left.

2.2 Search Algorithms

2.2.1 Breadth-First Search (BFS)

BFS is an **uninformed** search strategy that always expands the shallowest unexpanded node first, implemented with a FIFO deque. This tier-by-tier expansion guarantees[4]:

- **Completeness**: BFS finds a solution if one exists at any finite depth.
- **Optimality**: With uniform step costs, the first goal reached is via the shortest path.
- **Time complexity**: $O(b^d)$, where b is the branching factor and d is solution depth.
- **Space complexity**: $O(b^d)$, the entire frontier tier must be stored.

Algorithm 1 Breadth-First Search (BFS)

```
frontier ← Deque([start])
came_from[start] ← None
while frontier is not empty do
    current ← frontier.popleft()
    if current = goal then break
    end if
    for each open neighbour of current in maze_map do
        if neighbour ∉ came_from then
            came_from[neighbour] ← current
            frontier.append(neighbour)
        end if
    end for
end while
return reconstruct_path(came_from, goal)
```

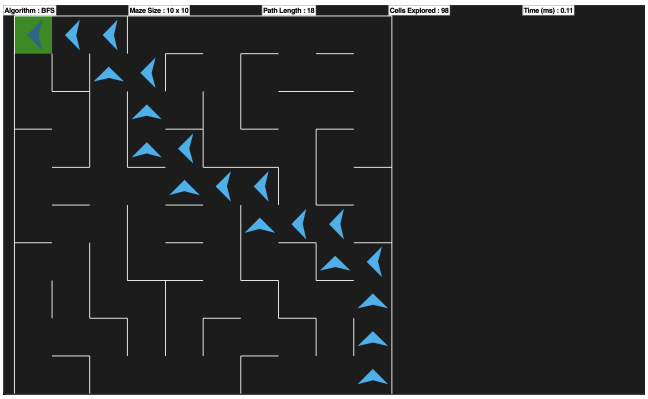


Figure 2: BFS traversal on a 10×10 maze, expanding nodes layer-by-layer.

2.2.2 Depth-First Search (DFS)

DFS always expands the deepest node first using a LIFO stack, diving as far as possible before backtracking[2;4]:

- **Completeness**: Complete with a visited set in finite spaces, but not in infinite-depth spaces.
- **Optimality**: *Not* optimal, returns the first path found, regardless of length.
- **Time complexity**: $O(b^m)$, where m is maximum depth.
- **Space complexity**: $O(bm)$, linear in depth, DFS's key advantage over BFS.

Algorithm 2 Depth-First Search (DFS)

```
stack ← [start]
came_from[start] ← None
while stack is not empty do
    current ← stack.pop()
    if current = goal then break
    end if
    for each open neighbour of current in maze_map do
        if neighbour ∉ came_from then
            came_from[neighbour] ← current
            stack.push(neighbour)
        end if
    end for
end while
return reconstruct_path(came_from, goal)
```

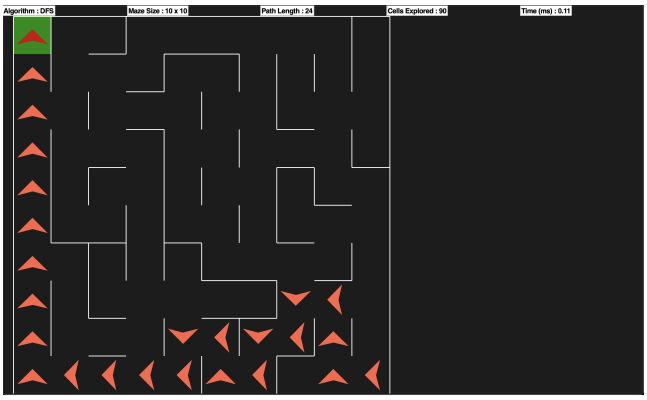


Figure 3: DFS traversal on a 10×10 maze, following a single deep branch before backtracking.

2.2.3 A* Search

A* is an **informed** search algorithm that combines accumulated cost $g(n)$ with a heuristic estimate $h(n)$ to guide expansion via $f(n) = g(n) + h(n)$. It uses a min-heap ordered by $f(n)$ and terminates when the goal is *dequeued* (not merely enqueued)[2;3].

Admissibility and optimality. A* is optimal when $h(n)$ is **admissible**: $h(n) \leq h^*(n)$ for all n[3]. For graph search, the stronger condition of **consistency** ($h(n) \leq \text{cost}(n, n') + h(n')$) ensures f-values are non-decreasing along any path[2;7].

Heuristic dominance. If $h_2(n) \geq h_1(n)$ for all n (both admissible), then h_2 *dominates* h_1 and yields fewer node expansions[7].

Two heuristics.

- **Manhattan**: $h(n) = |\Delta r| + |\Delta c|$, tight lower bound on 4-directional grids.

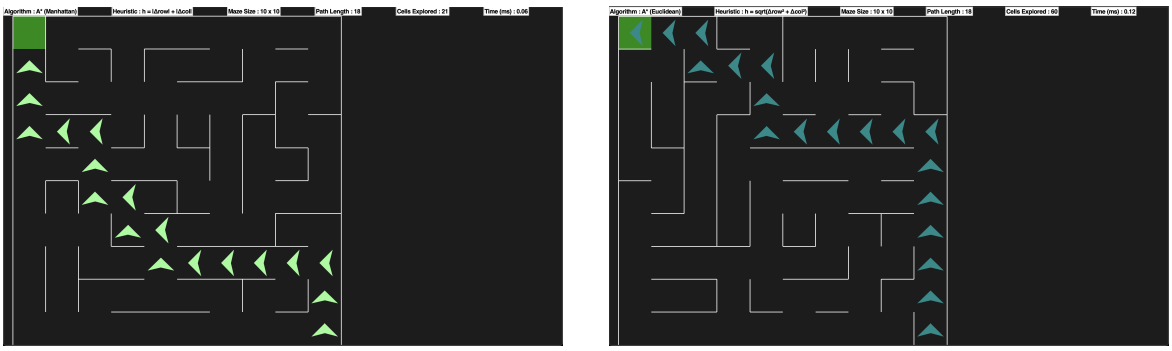
- **Euclidean**: $h(n) = \sqrt{(\Delta r)^2 + (\Delta c)^2}$, strictly weaker on 4-directional grids since $h_{\text{euclid}} \leq h_{\text{manhattan}}$ always holds.

Both are admissible and consistent on unit-cost grids, guaranteeing optimal paths[3].

Weighted A* (A*W). When cells carry variable costs $w \in \{1, \ldots, 9\}$, A*W uses $g(n) = g(\text{parent}) + w(\text{cell})$ instead of unit cost, finding cost-optimal rather than step-optimal paths. The Manhattan heuristic remains admissible because $\min(w) = 1$.

Algorithm 3 A* Search (heuristic h, optional cell weights w)

```
g[start] ← 0
open ← MinHeap({(h(start, goal), start)})
came_from[start] ← None
while open is not empty do
    (_, current) ← open.pop_min()
    if current = goal then break
    end if
    for each open neighbour of current in maze_map do
        cost ← w[neighbour] if weighted else 1
        g' ← g[current] + cost
        if g' < g.get(neighbour, ∞) then
            g[neighbour] ← g'
            f ← g' + h(neighbour, goal)
            open.push(f, neighbour)
            came_from[neighbour] ← current
        end if
    end for
end while
return reconstruct_path(came_from, goal)
```



(a) A*(Manhattan) on 10×10 **(b)** A*(Euclidean) on 10×10

Figure 4: A* with Manhattan (left) and Euclidean (right) heuristics. Manhattan's tighter bound produces a more focused search.

2.3 Markov Decision Processes

From deterministic search to stochastic planning. The search algorithms above assume a deterministic, fully observable environment. MDPs generalise this to **stochastic** settings where actions may have uncertain outcomes. The Markov property ensures that only the current state and action matter for transitions[8;9].

MDP formulation. The state space S contains all N^2 cells, with actions $A = \{N, S, E, W\}$. The transition function implements noise $= 0.2$: the intended move succeeds with probability 0.8; each perpendicular direction gets 0.1. The reward is $R_{\text{goal}} = +1.0$ with a living penalty $R = -0.04$ and discount $\gamma = 0.9$[5].

Optimal policy. An MDP yields a policy $\pi^* : S \to A$ for *every* state, satisfying the Bellman optimality equation:

$$V^*(s) = \max_{a \in A} \sum_{s'} T(s' \mid s, a) \left[R(s) + \gamma V^*(s') \right]. \tag{1}$$

2.3.1 Value Iteration (MDP-VI)

VI solves the Bellman equation by successive approximation[5;8]:

$$V_{k+1}(s) = R(s) + \gamma \max_{a \in A(s)} \sum_{s'} T(s'|s, a) \, V_k(s'). \tag{2}$$

Convergence is guaranteed with $\gamma < 1$. We stop when $\max_s |V_{k+1}(s) - V_k(s)| < 10^{-6}$ or after 500 iterations. Each iteration costs $O(|S|^2|A|)$; the policy is extracted by one-step lookahead[2].

2.3.2 Policy Iteration (MDP-PI)

PI alternates between two phases[6;8]:

1. **Policy Evaluation**: compute V^π by iterating the fixed-policy Bellman system (up to 200 inner sweeps):

$$V^\pi(s) = R(s) + \gamma \sum_{s'} T(s' \mid s, \pi(s)) \, V^\pi(s'). \tag{3}$$

2. **Policy Improvement**: update $\pi'(s) \leftarrow \arg\max_a \sum_{s'} T(s'|s, a) \, V^\pi(s')$.

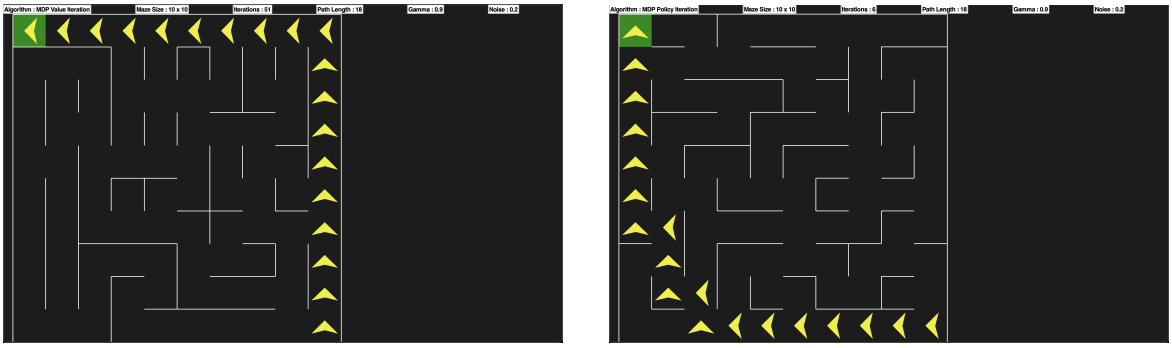
PI stops when no state changes its action. Because the policy often stabilises before values fully converge, PI needs fewer outer iterations than VI, though the inner evaluation adds cost[2].

Algorithm 4 MDP Value Iteration (MDP-VI)

```
Initialise V(s) ← 0 for all s; V(goal) ← 1
for k = 1, 2, ..., max_iter do
    δ ← 0
    for each state s ∈ S \ {goal} do
        V'(s) ← R(s) + γ max_a Σ_{s'} T(s'|s, a) V(s')
        δ ← max(δ, |V'(s) − V(s)|)
    end for
    V ← V'
    if δ < ϵ then break
    end if
end for
π(s) ← arg max_a Σ_{s'} T(s'|s, a) V(s') for all s
return π, extract_path(π, start, goal)
```

Algorithm 5 MDP Policy Iteration (MDP-PI)

```
Initialise π(s) ← arbitrary action for all s
Initialise V(s) ← 0 for all s; V(goal) ← 1
repeat                                                  ▷ Policy Evaluation
    for j = 1, ..., eval_iter do
        for each state s ∈ S \ {goal} do
            V(s) ← R(s) + γ Σ_{s'} T(s'|s, π(s)) V(s')
        end for
    end for                                             ▷ Policy Improvement
    stable ← True
    for each state s ∈ S \ {goal} do
        a* ← arg max_a Σ_{s'} T(s'|s, a) V(s')
        if a* ≠ π(s) then π(s) ← a*; stable ← False
        end if
    end for
until stable
return π, extract_path(π, start, goal)
```



(a) MDP Value Iteration path

(b) MDP Policy Iteration path

Figure 5: MDP-VI (left) and MDP-PI (right) on a 10×10 maze. Both compute a full policy over all states before extracting the path.

3 Experimental Setup

Maze sizes. Six sizes: 5×5, 10×10, 15×15, 25×25, 50×50, and 100×100 (25 to 10,000 cells).

Scenarios.

- **Unweighted**: All edges have unit cost. All six algorithms are compared.
- **Weighted**: Cells carry random costs $w \in \{1, \ldots, 9\}$. Search algorithms plus Weighted A* (A*W) variants are compared.
- **Stochastic**: A*'s fixed path vs MDP's adaptive policy under varying noise levels (0–0.4), measuring goal-arrival rate over 1,000 Monte Carlo trials.

Metrics.

- **Path length**: steps in the solution path.
- **Path cost** (weighted only): sum of cell weights along the path.
- **Cells explored**: nodes dequeued from the open structure (search only).
- **Solver time**: wall-clock time via `time.perf_counter()`.
- **Iterations** (MDP only): outer Bellman sweeps / policy improvement steps.

Path quality metric. The optimal lower bound $\text{OLB} = 2(N-1)$ gives the minimum path in a wall-free grid. The ratio path_length/OLB measures distance from ideal (1.0 = perfect).

Time complexity fitting. A power law $T(N) \propto N^k$ is fitted via log-log linear regression to estimate the empirical complexity exponent k.

MDP-VI path anomaly. At $N = 100$, MDP-VI's path extraction returned only 4 steps due to a cycle in the stochastic policy near the start cell. The `_extract_path` method uses a visited set that terminates early on cycles. These data points are excluded from path-quality analysis. MDP-PI does not exhibit this issue.

4 Results: Search Algorithm Comparison

4.1 Unweighted Maze, Raw Data

Table 1 summarises the search algorithm results on unweighted mazes.

Table 1: Search algorithms on unweighted mazes: path length, cells explored, and solver time.

Algorithm	**5**	**10**	**15**	**25**	**50**	**100**
Path length (steps)						
BFS	8	18	38	52	104	212
DFS	10	54	60	128	702	2380
A*(Manhattan)	8	18	38	52	104	212
A*(Euclidean)	8	18	38	52	104	212
Cells explored						
BFS	25	100	225	625	2487	9999
DFS	22	65	79	224	1710	7455
A*(Manhattan)	9	21	217	280	823	6115
A*(Euclidean)	14	63	219	507	1937	9049
Solver time (ms)						
BFS	0.028	0.066	0.149	0.378	1.457	6.155
DFS	0.020	0.047	0.061	0.151	1.051	5.031
A*(Manhattan)	0.017	0.032	0.228	0.336	1.004	8.473
A*(Euclidean)	0.023	0.066	0.200	0.516	1.916	9.429

4.2 Path Quality

BFS, A*(Manhattan), and A*(Euclidean) all find optimal paths of identical length at every size. DFS, by contrast, returns paths that grow dramatically longer: at $N = 100$, DFS produces a path of **2,380 steps** compared to BFS/A*'s 212 — an **11.2**$\times$ inflation. This confirms DFS's fundamental lack of optimality guarantees.

4.3 Exploration Efficiency

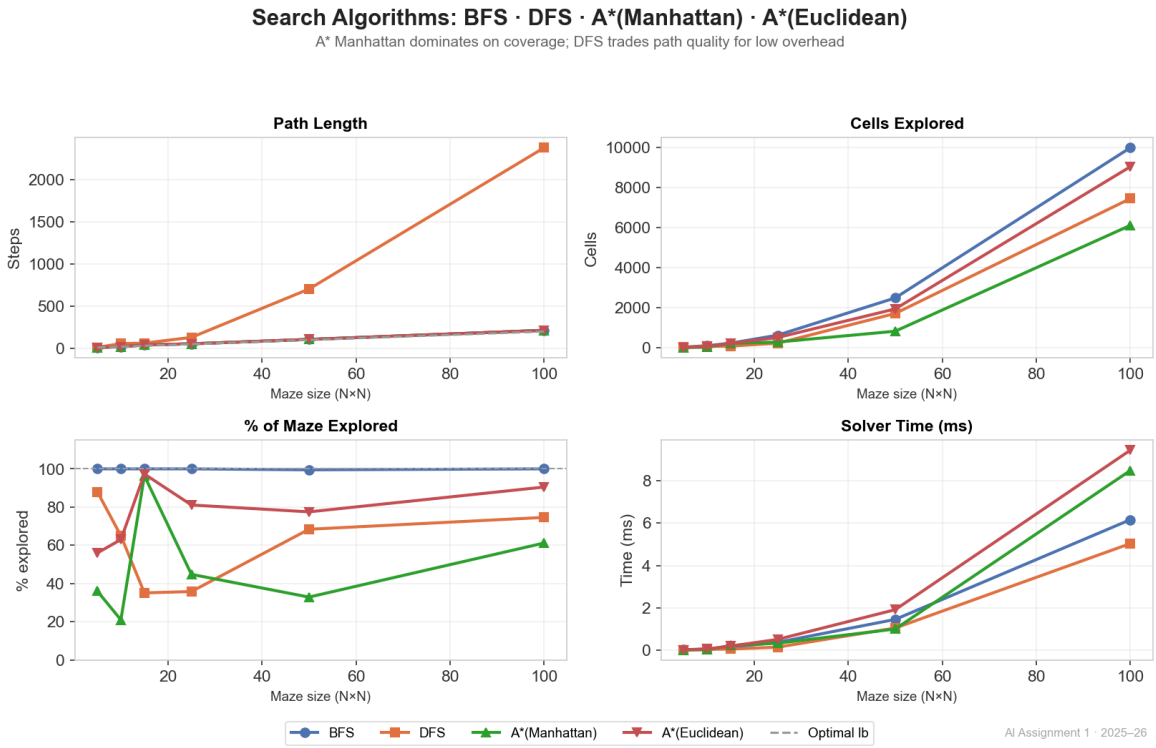


Figure 6: Four-panel comparison of all search algorithms across path length, cells explored, % of maze explored, and solver time.

The most striking result is in exploration efficiency. At $N = 100$, BFS explores **9,999 of 10,000 cells** (effectively the entire maze), while A* Manhattan explores only **6,115 cells** — a **39% reduction**. A* Euclidean explores 9,049 cells, significantly more than Manhattan, confirming Manhattan's *dominance* on 4-directional grids ($h_{\text{euclid}} \leq h_{\text{manhattan}}$ always holds).

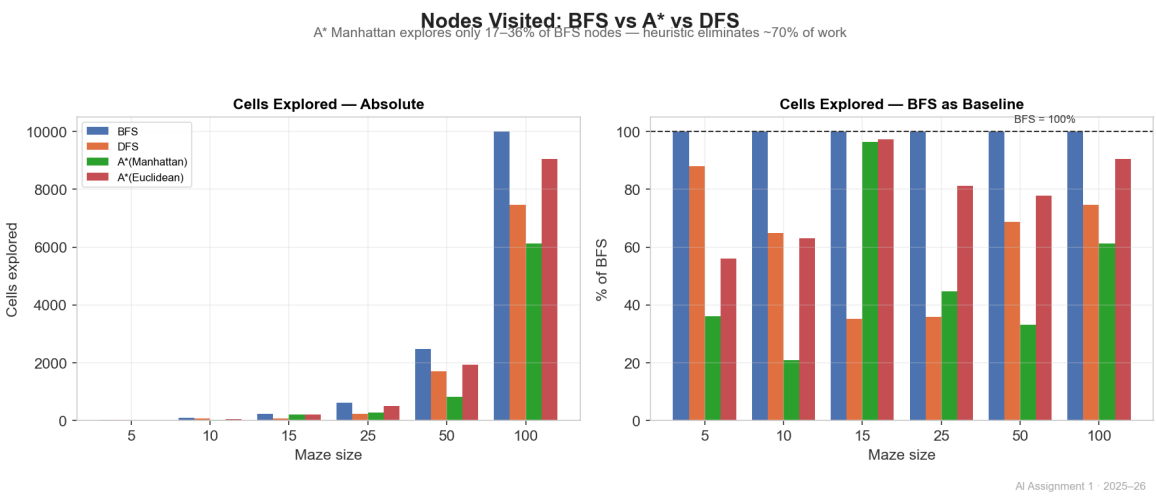


Figure 7: Left: absolute cells explored. Right: relative to BFS (BFS = 100%).

4.4 Wall-Clock Time and Per-Node Overhead

DFS is the **fastest** algorithm in wall-clock time despite producing the worst paths (5.0 ms at $N = 100$ vs BFS's 6.2 ms and A* Manhattan's 8.5 ms). This apparent paradox is explained by per-node overhead: DFS uses $O(1)$ stack operations (`list.pop()`/`append()`), while A* uses $O(\log n)$ heap operations (`heappush`/`heappop`) plus heuristic computation at every expansion. The heap overhead per node outweighs A*'s advantage in exploring fewer nodes.

4.5 Weighted Maze, Search Algorithm Comparison

When cells carry variable traversal costs ($w \in \{1, \ldots, 9\}$), the shortest path in steps is no longer the cheapest in cost. We add **Weighted A*** (A*W), which uses $g(n) = g(\text{parent}) + w(\text{cell})$, to find cost-optimal paths.

Table 2: Step-optimal vs cost-optimal routing on weighted mazes. A*W trades extra steps for lower cost.

	15×15		50×50		100×100	
Algorithm	**Steps**	**Cost**	**Steps**	**Cost**	**Steps**	**Cost**
BFS	38	182	104	554	212	1013
A*(Manhattan)	38	182	104	551	212	1023
A*(Euclidean)	38	180	104	546	212	1081
A*W(Manhattan)	38	176	112	495	218	911
A*W(Euclidean)	40	176	112	495	218	911
DFS	60	273	702	3637	2380	11887

Cost savings from weight-aware routing grow with maze size: **3.3%** at $N = 15$, **10.6%** at $N = 50$, and **10.1%** at $N = 100$ (A*W vs BFS). A*W takes slightly longer paths in steps (218 vs 212 at $N = 100$) but achieves a cost of 911 vs BFS's 1,013 — a meaningful saving. The `loopPercent=50` setting is critical: in a perfect maze only one path exists, making cost-vs-step optimality meaningless.

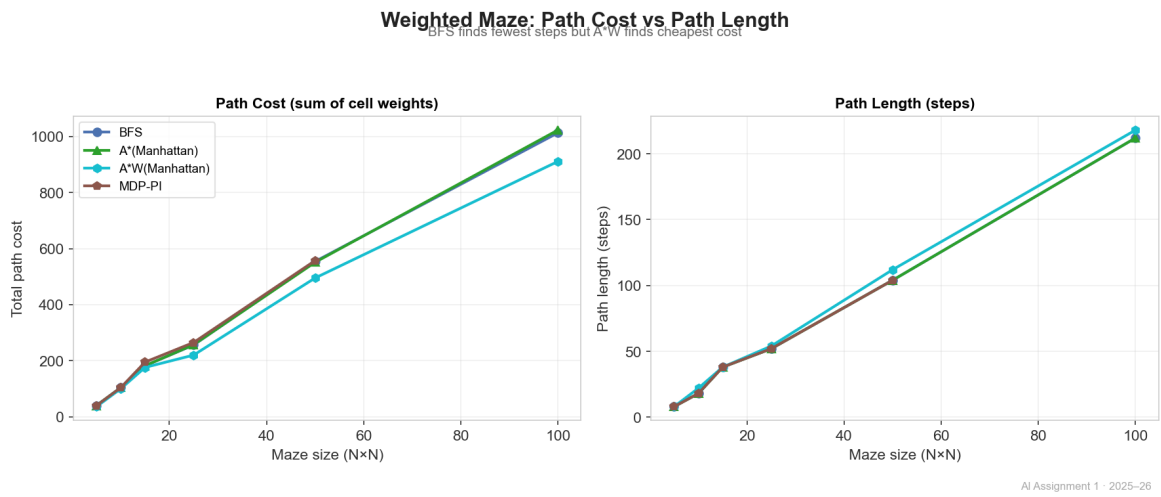


Figure 8: Weighted maze: path cost (left) vs path length (right). BFS minimises steps; A*W minimises cost.

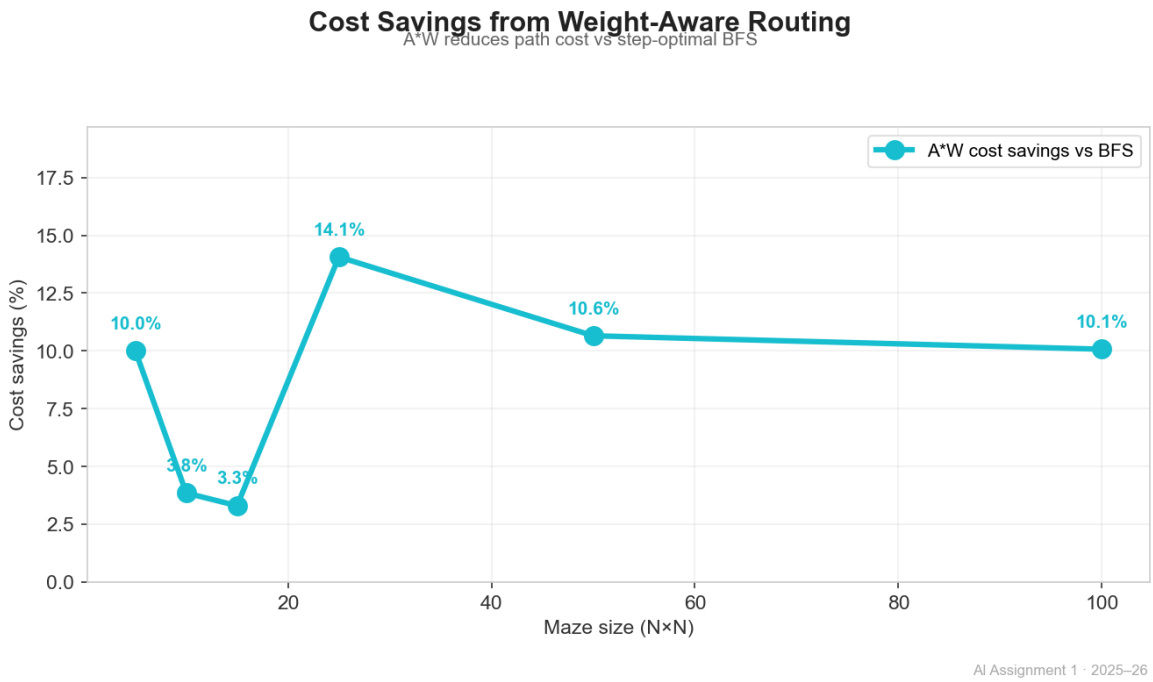


Figure 9: Percentage cost savings of A*W over BFS across maze sizes.

5 Results: MDP vs Search Comparison (Unweighted)

This section compares MDP-VI and MDP-PI against the search algorithms on **unweighted** mazes, where the fundamental trade-offs between the two paradigms are most clearly visible.

5.1 Raw Benchmark Data

Table 3: All algorithms on unweighted mazes. "—" indicates excluded MDP-VI path anomaly at $N = 100$.

Algorithm	**5**	**10**	**15**	**25**	**50**	**100**
Path length (steps)						
BFS	8	18	38	52	104	212
DFS	10	54	60	128	702	2380
A*(Manhattan)	8	18	38	52	104	212
A*(Euclidean)	8	18	38	52	104	212
MDP-VI	8	18	38	52	104	—
MDP-PI	8	18	38	52	104	212
Solver time (ms)						
BFS	0.028	0.066	0.149	0.378	1.457	6.155
DFS	0.020	0.047	0.061	0.151	1.051	5.031
A*(Manhattan)	0.017	0.032	0.228	0.336	1.004	8.473
A*(Euclidean)	0.023	0.066	0.200	0.516	1.916	9.429
MDP-VI	2.722	19.351	60.041	206.605	1005.552	3988.818
MDP-PI	3.617	28.368	110.418	393.637	1750.055	10446.592
MDP iterations to convergence						
MDP-VI	31	50	71	84	102	102
MDP-PI	3	6	11	13	15	60

5.2 Path Quality: MDP Matches Search

When MDP-VI successfully extracts a path (sizes 5–50), it produces **identical path lengths** to BFS and A*. MDP-PI matches search at all sizes including $N = 100$ (212 steps). This demonstrates that the stochastic MDP formulation with $\gamma = 0.9$ and noise $= 0.2$ still converges to the optimal deterministic path when the environment is actually deterministic during path extraction.

5.3 The Speed Gap

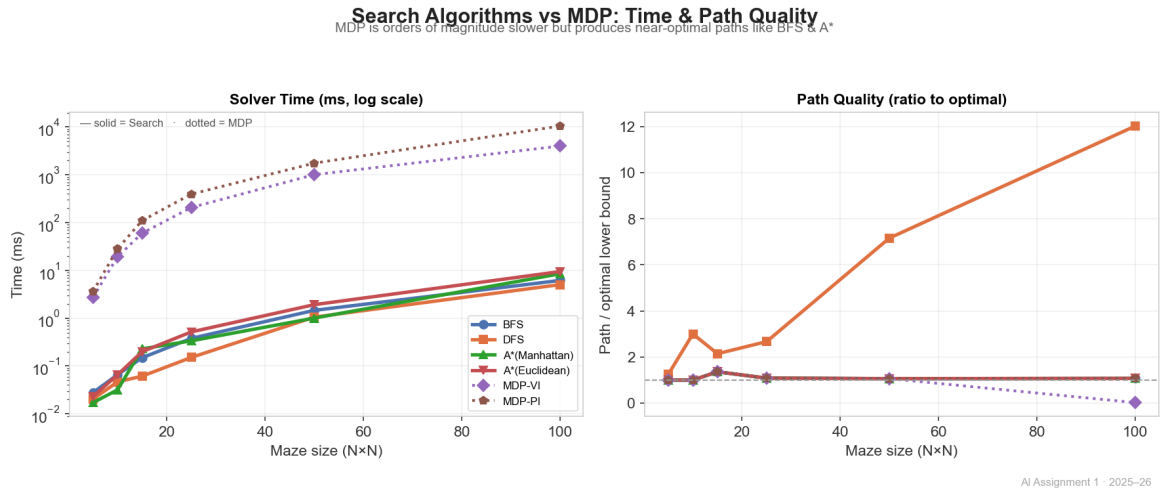


Figure 10: All algorithms on common axes, time (log scale, left) and path quality (right). Solid = search; dashed = MDP.

The performance gap is dramatic. At $N = 100$: A* Manhattan solves the maze in **8.5 ms**, MDP-VI in **3,989 ms** (**470× slower**), and MDP-PI in **10,447 ms** (**1,230× slower**). Both paradigms achieve comparable path quality, but MDP pays a massive computational premium.

The reason is fundamental: search algorithms explore *only the cells needed* to find a single start-to-goal path, while MDP must compute value estimates for *every cell in the maze* (N^2 states) over multiple iterations.

5.4 MDP-VI vs MDP-PI

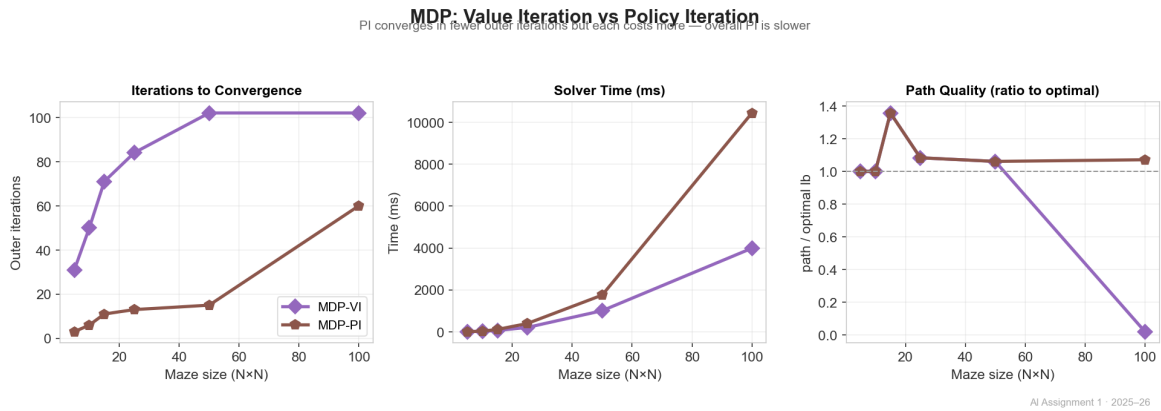


Figure 11: MDP-VI vs MDP-PI: iterations, solver time, and path quality.

MDP-PI converges in far fewer outer iterations (3–60 vs 31–102 for VI), but is **consistently slower** in wall-clock time: 10,447 ms vs 3,989 ms at $N = 100$. Each PI iteration runs an expensive inner policy-evaluation loop (up to 200 Bellman sweeps), making the per-iteration cost much higher. This illustrates that iteration count alone does not determine computational cost; VI is the more practical choice for this problem class.

5.5 Time Complexity Analysis

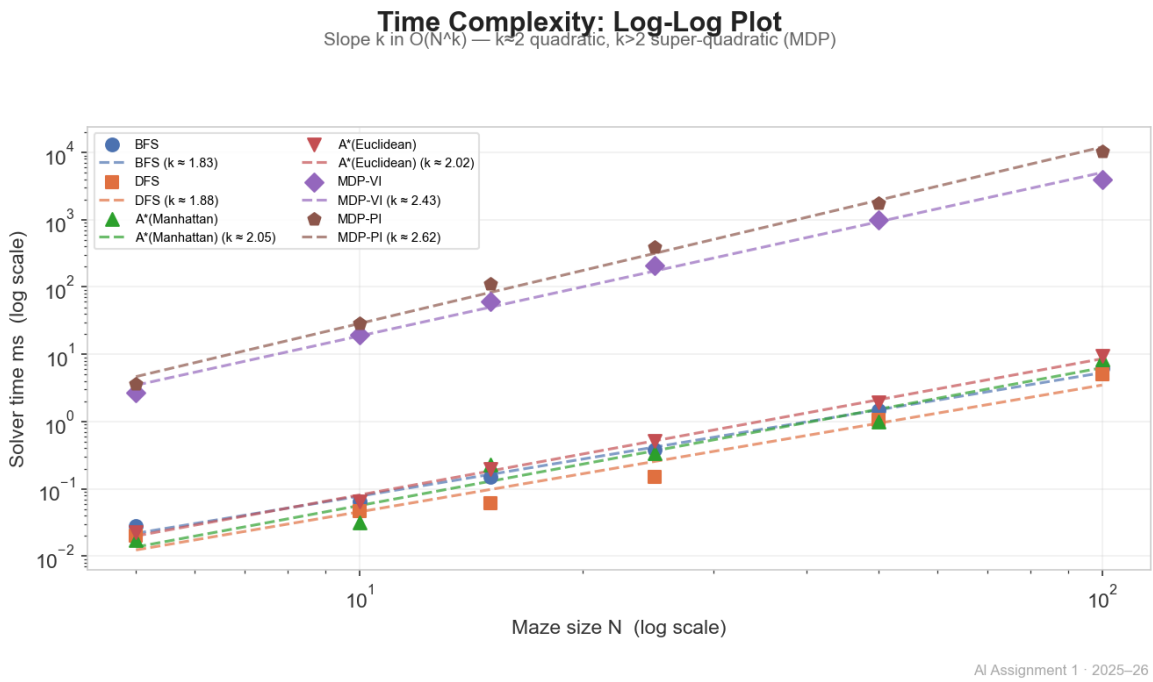


Figure 12: Log-log time plot with fitted power-law lines. Slope k in $O(N^k)$.

Table 4: Empirical time-complexity exponents from power-law fitting ($T \propto N^k$).

Algorithm	Classification	Exponent k
BFS	Sub-quadratic	≈1.8
DFS	Sub-quadratic	≈1.9
A*(Manhattan)	Quadratic	≈2.1
A*(Euclidean)	Quadratic	≈2.0
MDP-VI	Super-quadratic	≈2.4
MDP-PI	Super-quadratic	≈2.6

BFS and DFS both scale sub-quadratically ($k \approx 1.8$–1.9). A* Manhattan and Euclidean scale quadratically ($k \approx 2.0$–2.1); despite exploring fewer cells, the heap overhead per node raises the empirical exponent. Both MDP methods are **super-quadratic** ($k > 2$) because the Bellman iteration count itself grows with maze size, yielding $k \approx 2.4$–2.6.

5.6 When Does MDP Make Sense?

Despite being dramatically slower for single queries, MDP offers three fundamental advantages that search cannot provide.

1. Policy completeness. Search produces one start-to-goal path covering 212 states at $N = 100$; MDP computes a policy $\pi(s)$ for all 10,000 states. If the start position changes, A* must recompute; the MDP policy is already precomputed.

2. Amortised multi-query cost. MDP pays its cost once; search recomputes per query. The crossover where MDP becomes cheaper occurs at ≈470 queries for $N = 100$ (A^* Manhattan vs MDP-VI) and fewer for larger mazes.

3. Stochastic robustness. This is MDP's decisive advantage. On a 50×50 maze, A^*'s fixed path drops to **0% success at noise** ≥ 0.05, while MDP maintains **100% at all noise levels up to 0.4** (Figure 13). The MDP policy is reactive — at each time step, it selects the best action for the state the agent *actually reaches*, rather than following a predetermined sequence.

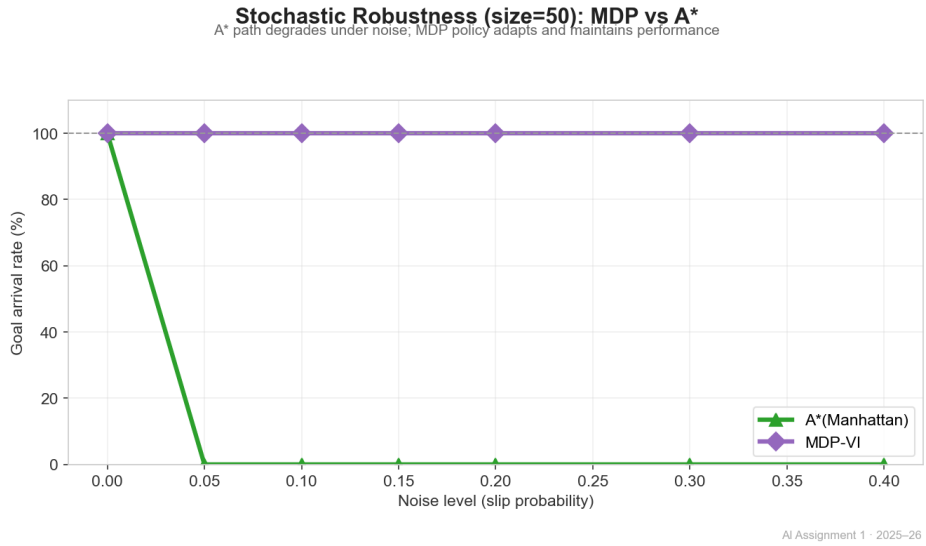


Figure 13: Goal-arrival rate under varying noise (50×50 maze, 1,000 trials). A^*'s fixed path degrades instantly; MDP's adaptive policy remains robust.

Table 5: Decision guide: when to use search vs MDP.

Dimension	Search (A^*)	MDP (VI / PI)
Single query	470–1,230× faster	Wastes computation
Multi-query (>470)	Recomputes each time	Amortises precomputation
Deterministic env.	Optimal and efficient	Optimal but over-engineered
Stochastic env.	Brittle (0% at noise=0.05)	Robust (100% at all noise)
Full state coverage	Single path only	Policy for all N^2 states

6 Conclusions

This study benchmarks six algorithms on maze solving across six grid sizes in both unweighted and weighted scenarios. The key findings are:

1. **A^* with Manhattan distance** is the best single-query algorithm for unweighted mazes: it finds optimal paths while exploring 39% fewer cells than BFS, completing a 100×100 maze in 8.5 ms.

2. **BFS** guarantees optimal paths and is the simplest correct algorithm, but explores the entire maze at scale (≈100% of cells at $N = 100$).
3. **DFS** is fastest in wall-clock time (5.0 ms at $N = 100$) due to $O(1)$ stack operations, but produces paths **11.2× longer** than optimal. It should only be used when path quality is irrelevant and memory is constrained.
4. **Manhattan dominates Euclidean** on 4-directional grids. Euclidean explores 9,049 cells at $N = 100$ vs Manhattan's 6,115 — a 48% increase for identical path quality.
5. **Weighted A*** achieves 10% cost savings over step-optimal algorithms on weighted mazes, trading slightly longer paths (218 vs 212 steps) for significantly lower total cost (911 vs 1,013 at $N = 100$).
6. **MDP is fundamentally slower**: MDP-VI is 470× and MDP-PI is 1,230× slower than A* at $N = 100$, with super-quadratic scaling ($k \approx 2.4$–2.6 vs ≈1.8–2.1 for search).
7. **MDP-VI outperforms MDP-PI** in practice: despite PI's fewer outer iterations (60 vs 102), VI is 2.6× faster at $N = 100$ because PI's inner evaluation loop is expensive.
8. **MDP's stochastic robustness is its decisive advantage**: under noise = 0.05, A*'s fixed path fails entirely while MDP maintains 100% goal-arrival rate. For stochastic or multi-query environments, the computational cost is justified.

The choice of algorithm depends on the use case: A* Manhattan for single deterministic queries, Weighted A* for cost-sensitive environments, and MDP for multi-query or stochastic settings where a complete precomputed policy provides ongoing value.

References

[1] M. Alibhai, *pyamaze: A Python package for generating and solving mazes*. GitHub, 2022. github.com/MAN1986/pyamaze

[2] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Prentice Hall, 2010. aima.cs.berkeley.edu

[3] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968. doi:10.1109/TSSC.1968.300136

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009. mitpress.mit.edu/9780262046305

[5] R. Bellman, "A Markovian decision process," *Journal of Mathematics and Mechanics*, vol. 6, no. 5, pp. 679–684, 1957. doi:10.1512/iumj.1957.6.56038

[6] R. A. Howard, *Dynamic Programming and Markov Processes*. MIT Press, 1960. archive.org | Howard 1960

[7] J. Pearl, *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984. dl.acm.org/doi/book/10.5555/390

[8] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994. doi:10.1002/9780470316887

[9] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018. incompleteideas.net/book

[10] V. Bellot *et al.*, "How to generate perfect mazes?" *Information Sciences*, vol. 572, pp. 444–459, 2021. doi:10.1016/j.ins.2021.04.091

A Appendix

This appendix presents key Python functions that implement the maze generation, search algorithms, and MDP solvers discussed in the report.

A.1 Maze Generation and Neighbour Lookup

Mazes are generated using `pyamaze` and cached to CSV for reproducibility. All solvers share a common direction map and neighbour function.

```python
DIRECTION_MAP = {
    'N': (-1,  0),
    'S': ( 1,  0),
    'E': ( 0,  1),
    'W': ( 0, -1),
}

def get_or_create_maze(size, loop_percent=0):
    csv_path = os.path.join('temp', f'maze_{size}.csv')
    m = maze(size, size)
    if os.path.exists(csv_path):
        m.CreateMaze(loadMaze=csv_path)
    else:
        m.CreateMaze(loopPercent=loop_percent)
        _save_maze_csv(m.maze_map, csv_path)
    return m
```

Listing 1: Direction map and maze generation (`utils/maze_utils.py`).

```python
class Solver:
    def __init__(self, m):
        self.m     = m
        self.start = (m.rows, m.cols)
        self.goal  = (1, 1)

    def neighbours(self, cell):
        r, c = cell
        result = []
        for d, (dr, dc) in DIRECTION_MAP.items():
            if self.m.maze_map[cell][d]:
                result.append((r + dr, c + dc))
        return result

    def reconstruct_path(self, came_from):

        path, cell = {}, self.goal
        while cell != self.start:
            path[came_from[cell]] = cell
            cell = came_from[cell]
        return path
```

Listing 2: Solver base class, neighbour lookup and path reconstruction (`solvers/base.py`).

A.2 Breadth-First Search (BFS)

```
class BFSSolver(Solver):
    def solve(self):
        q = deque([self.start])
        came_from = {self.start: None}
        cells_processed = 0

        while q:
            current = q.popleft()
            cells_processed += 1
            if current == self.goal:
                break
            for neighbour in self.neighbours(current):
                if neighbour not in came_from:
                    came_from[neighbour] = current
                    q.append(neighbour)

        return self.reconstruct_path(came_from), \
               {'cells_explored': cells_processed}
```

Listing 3: BFS, guarantees shortest path via FIFO queue (`solvers/bfs.py`).

A.3 Depth-First Search (DFS)

```
class DFSSolver(Solver):
    def solve(self):
        stack     = [self.start]
        came_from = {self.start: None}
        closed    = set()
        cells_processed = 0

        while stack:
            current = stack.pop()
            if current in closed:
                continue
            closed.add(current)
            cells_processed += 1
            if current == self.goal:
                break
            for neighbour in self.neighbours(current):
                if neighbour not in came_from:
                    came_from[neighbour] = current
                    stack.append(neighbour)

        return self.reconstruct_path(came_from), \
               {'cells_explored': cells_processed}
```

Listing 4: DFS, dives deep via LIFO stack; no optimality guarantee (`solvers/dfs.py`).

A.4 A* Search

```python
def manhattan(a, b):
    """h = |dr| + |dc|  -- tight on 4-directional grids."""
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def euclidean(a, b):
    return math.sqrt((a[0] - b[0])**2 + (a[1] - b[1])**2)
```

Listing 5: Heuristic functions (`utils/maze_utils.py`).

```python
class AStarSolver(Solver):
    def __init__(self, m, heuristic='manhattan', weights=None):
        super().__init__(m)
        self.h       = HEURISTICS[heuristic]
        self.weights = weights

    def solve(self):
        open_set  = [(self.h(self.start, self.goal), self.start)]
        came_from = {self.start: None}
        g_score   = {self.start: 0}
        closed    = set()
        cells_processed = 0

        while open_set:
            _, current = heapq.heappop(open_set)
            if current in closed:
                continue
            closed.add(current)
            cells_processed += 1
            if current == self.goal:
                break
            for nb in self.neighbours(current):
                if nb in closed:
                    continue
                cost = self.weights[nb] if self.weights else 1
                tentative_g = g_score[current] + cost
                if nb not in g_score or tentative_g < g_score[nb]:
                    g_score[nb]   = tentative_g
                    f = tentative_g + self.h(nb, self.goal)
                    came_from[nb] = current
                    heapq.heappush(open_set, (f, nb))

        return self.reconstruct_path(came_from), \
               {'cells_explored': cells_processed}
```

Listing 6: A* Search with optional cell weights (`solvers/astar.py`).

A.5 MDP, Stochastic Transition Model

```python
PERPENDICULAR = {
    'N': ['E', 'W'], 'S': ['E', 'W'],
    'E': ['N', 'S'], 'W': ['N', 'S'],
}

class MDPSolver(Solver):
    def __init__(self, m, gamma=0.9, noise=0.2,
                 living_reward=-0.04, theta=1e-6, max_iters=500):
        super().__init__(m)
        self.gamma, self.noise = gamma, noise
        self.living_reward     = living_reward
        self.theta, self.max_iters = theta, max_iters

    def _transition_probs(self, cell, action):
        nb = self.neighbours_dict(cell)
        outcomes = {}
        intended = nb.get(action, cell)
        outcomes[intended] = 1 - self.noise
        for perp in PERPENDICULAR[action]:
            slip = nb.get(perp, cell)
            outcomes[slip] = outcomes.get(slip, 0) + self.noise / 2
        return outcomes

    def _q_value(self, cell, action, V):
        q = self.living_reward
        for s_prime, prob in self._transition_probs(cell, action).items():
            q += prob * self.gamma * V[s_prime]
        return q
```

Listing 7: Stochastic transitions and Q-value computation (`solvers/mdp.py`).

A.6 MDP, Value Iteration

```python
    def solve(self):
        all_cells = list(self.m.maze_map.keys())
        V      = {c: 0.0 for c in all_cells}
        V[self.goal] = 1.0
        policy = {c: None for c in all_cells}

        for i in range(self.max_iters):
            delta = 0.0
            new_V = dict(V)
            for cell in all_cells:
                if cell == self.goal:
                    continue
                best_val, best_a = self._best_action(cell, V)
                new_V[cell]  = best_val
```

```
                policy[cell] = best_a
                delta = max(delta, abs(new_V[cell] - V[cell]))
            V = new_V
            if delta < self.theta:
                break

        path = self._extract_path(policy, V)
        return path, {'iterations': i+1, 'V': V}
```

Listing 8: Value Iteration, Bellman update until convergence (`solvers/mdp.py`).

A.7 MDP, Policy Iteration

```
class MDPPolicyIterationSolver(MDPSolver):

    def _evaluate_policy(self, policy, V):
        all_cells = list(self.m.maze_map.keys())
        for _ in range(self.max_eval_iters):
            delta, new_V = 0.0, dict(V)
            for cell in all_cells:
                if cell == self.goal: continue
                new_V[cell] = self._q_value(cell, policy[cell],
                    V)
                delta = max(delta, abs(new_V[cell] - V[cell]))
            V = new_V
            if delta < self.theta: break
        return V

    def solve(self):
        all_cells = list(self.m.maze_map.keys())
        policy = {c: 'N' for c in all_cells}
        V = {c: 0.0 for c in all_cells}; V[self.goal] = 1.0

        while True:
            V = self._evaluate_policy(policy, V)
            stable = True
            for cell in all_cells:
                if cell == self.goal: continue
                _, best_a = self._best_action(cell, V)
                if best_a != policy[cell]:
                    policy[cell] = best_a
                    stable = False
            if stable: break

        return self._extract_path(policy, V), {'iterations': i}
```

Listing 9: Policy Iteration, evaluate then improve (`solvers/mdp.py`).